

Trees

16.1 Regression Trees

Regression trees are similar to additive models in that they represent a compromise between the linear model and the completely nonparametric approach. Tree methodology has roots in both the statistics and computer science literature. A precursor to current methodology was CHAID developed by Morgan and Sonquist (1963) although the book by Breiman et al. (1984) introduced the main ideas to statistics. Concurrently, tree methodology was developed in machine learning starting in the 1970s — see Quinlan (1993) for an overview.

Most statistical work starts from the specification of a model. The model says how we believe the data is generated and contains both a systematic and a random component. The model is not completely specified and so we use the data to select a particular model by either estimating parameters or perhaps by fitting functions, as in our recent nonparametric approaches. This strategy has been effective in a wide range of situations. However, the insistence on specifying a model, right from the start, does limit statistics. It is often difficult to specify a model, particularly for larger and more complex datasets. Furthermore, it is often impractical to develop inferential methods for more complex statistical models.

Tukey (1977) advocated exploratory data analysis (EDA) in his book. Graphical and descriptive statistics can sometimes make the message of the data very clear or at least suggest a suitable form for the model. However, EDA is not a complete solution and sometimes we need definite predictions or conclusions.

Regression trees are an example of a statistical method that is best described by the algorithm used in their construction. One can uncover the implicit model underlying regression trees, but the algorithm is the true starting point. Any method of analysis should ultimately be judged on whether it successfully predicts or explains something. Statistical models may achieve this, but algorithmically based methods are also competitive. The distinction between algorithm-based and model-based methods is discussed in Breiman (2001b). In the computer science literature, tree methodology has been applied to decision tree problems where there is no stochastic structure and we simply want to build a rule for making the correct decision.

To grow a tree, we use the recursive partitioning regression algorithm:

1. Consider all partitions of the region of the predictors into two regions where the division is parallel to one of the axes. In other words, we partition a single predictor by choosing a point along the range of that predictor to make the split. It does

not matter exactly where we make the split between two adjacent points so there will be at most $(n - 1)p$ partitions to consider.

2. For each partition, we take the mean of the response in that partition. We then compute:

$$RSS(partition) = RSS(part_1) + RSS(part_2)$$

We then choose the partition that minimizes the residual sum of squares (RSS). We do need to consider many partitions, but the computations on each partition are simple, so that fit can be accomplished without excessive effort.

3. We now subpartition the partitions in a recursive manner. We only allow partitions within existing partitions and not across them. This means that the partitioning can be represented using a tree. There is no restriction preventing us from splitting the same variables consecutively.

For categorical predictors, it is possible to split on the levels of the factor. For an ordered factor with L levels, there are only $L - 1$ possible splits. For an unordered factor, there are $2^{L-1} - 1$ possible splits. Although this is a large number of possibilities as L grows, there is a way to limit the number that needs to be considered. Notice that there is no point in monotonely transforming a quantitative predictor as this will have no effect on the partitioning algorithm. Transforming the response will make a difference because it will change the computation of the RSS.

Missing values can be handled quite easily by tree methods. When we construct the tree, we may encounter missing values for a predictor when we are considering a split on that variable. We may simply exclude such points from the computation provided we weight appropriately. This approach is suitable for data where the observations are missing in a noninformative manner. If we believe the fact of being missing expresses some information, we might choose to treat missingness as an additional level of a factor. For continuous predictors, we could discretize the data into ranges so that it becomes a factor and then add missingness as an additional level. When we wish to predict the response for a new value with missing values, we can drop the prediction down through the tree until the missing values prevent us from going further. We can then use the mean value for that internal node.

Tree models are well suited to finding interactions. If we split on one variable and then split on another variable within the partitions of the first variable, we are finding an interaction between these two variables. As we construct further splits within splits, we are finding higher and higher order interactions. This may be a disadvantage as true high-order interactions are not common in reality. The MARS method discussed in Section 15.7 counteracts this by limiting the amount of interaction.

Trees are quite popular because the structure is easier for nontechnical people to understand. The term CART stands for Classification and Regression Trees and is also the name of a commercial software product.

We applied the regression tree methodology to study the relationship between atmospheric ozone concentration and meteorology in the Los Angeles Basin in 1976, as introduced in Section 15.1. A number of cases with missing variables have been removed for simplicity. The data were first presented by Breiman and Friedman (1985). We wish to predict the ozone level from the other predictors. We read in the data:

```
data(ozone, package="faraway")
```

An examination of the data reveals several nonlinear relationships indicating that a linear regression might not be appropriate without the addition of some transformations. Now fit a tree:

```
library(rpart)
```

```
(tmod <- rpart(O3 ~ ., ozone))
```

```
n= 330
```

```
node), split, n, deviance, yval
```

```
* denotes terminal node
```

```
1) root 330 21115.00 11.7760
  2) temp< 67.5 214 4114.30 7.4252
    4) ibh>=3573.5 108 689.63 5.1481 *
    5) ibh< 3573.5 106 2294.10 9.7453
      10) dpq< -9.5 35 362.69 6.4571 *
      11) dpq>=-9.5 71 1366.50 11.3660
        22) ibt< 159 40 287.90 9.0500 *
        23) ibt>=159 31 587.10 14.3550 *
  3) temp>=67.5 116 5478.40 19.8020
    6) ibt< 226.5 55 1276.80 15.9450
      12) humidity< 59.5 10 167.60 10.8000 *
      13) humidity>=59.5 45 785.64 17.0890 *
    7) ibt>=226.5 61 2646.30 23.2790
      14) doy>=306.5 8 398.00 16.0000 *
      15) doy< 306.5 53 1760.50 24.3770
        30) vis>=55 36 1149.90 22.9440 *
        31) vis< 55 17 380.12 27.4120 *
```

We see that the first split (nodes 2 and 3) is on temperature: 214 observations have temperatures less than 67.5 with a mean response value of 7.4, whereas 116 observations have temperatures greater than 67.5 with a mean response value of 20. The total RSS has been reduced from 21,115 to $4114 + 5478 = 9592$. The indentation describes the level of nesting within the tree. The children of node x are labeled as $2x$ and $2x + 1$. So for example, node 8 does not appear because node 4 was not split, even though there are more than 8 nodes in this tree. Much more substantial output can be obtained from `summary(tmod)`.

Although the relevant information can be gleaned from the text-based output, a graphical display is nicer as in Figure 16.1. In the first version of the plot, the depth of the branches is proportional to the reduction in error due to the split. The disadvantage is that the labels can be hard to read in lower parts of the tree where the reduction in error is much smaller. The second version of the plot uses a uniform spacing to allow more room for labeling:

```
plot(tmod)
```

```
text(tmod)
```

```
plot(tmod, compress=T, uniform=T, branch=0.4)
```

```
text(tmod)
```

We see that the first split on temperature produces a large reduction in the RSS. Some of the subsequent splits do not do much. The immediate message is that high

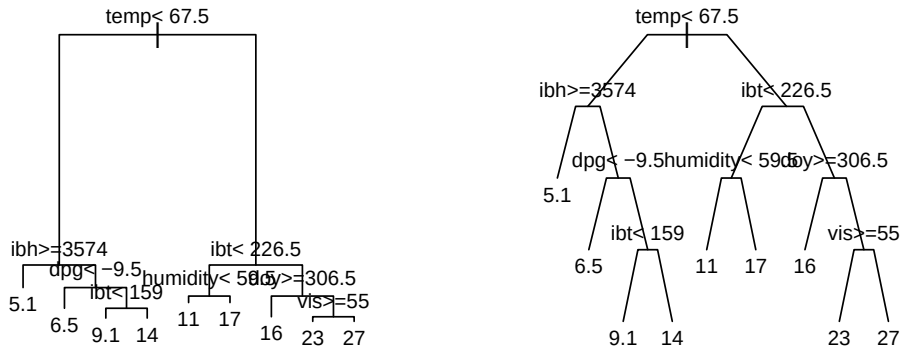


Figure 16.1 *Tree model for the ozone data. On the left, the depth of the branches is proportional to the improvement in fit. On the right, the depth is held constant to improve readability. If the logical condition at a node is true, follow the branch to the left.*

temperatures are associated with high ozone levels. A regression tree is a regression model, so diagnostics are worthwhile:

```
plot(jitter(predict(tmod)), residuals(tmod), xlab="Fitted", ylab="
  ↳ Residuals")
abline(h=0)
qqnorm(residuals(tmod))
qqline(residuals(tmod))
```

See Figure 16.2. There are no visible problems here. If nonconstant variance is observed, one might consider transforming the response. Trees are also somewhat sensitive to outliers as they are based on local means. Outliers may be observed in the QQ plot, but, as with linear models, they may conceal themselves and be influential on the fit. Suppose we wanted to predict the response for a new value — for example the median value in the dataset:

```
(x0 <- apply(ozone[, -1], 2, median))
  vh      wind humidity      temp      ibh      dpg      ibt      vis      doy
5760.0      5.0      64.0      62.0    2112.5      24.0     167.5     120.0     205.5
predict(tmod, data.frame(t(x0)))
1
14.355
```

You should be able to verify this prediction by following the splits down through the tree shown in Figure 16.1.

16.2 Tree Pruning

The recursive partitioning algorithm describes how to grow the tree, but what is the optimal size for the tree? The default form of `rpart` does restrict the size of the tree, but some intervention is probably necessary to select the best tree size.

One possibility, called a *greedy strategy*, is to keep partitioning until the reduction in overall cost (RSS for this type of tree) is not reduced by more than ϵ . However, it is difficult to set ϵ in a sensible way. Furthermore, a greedy strategy may stop too soon. For example, consider data laid out in Table 16.1: neither the horizontal nor

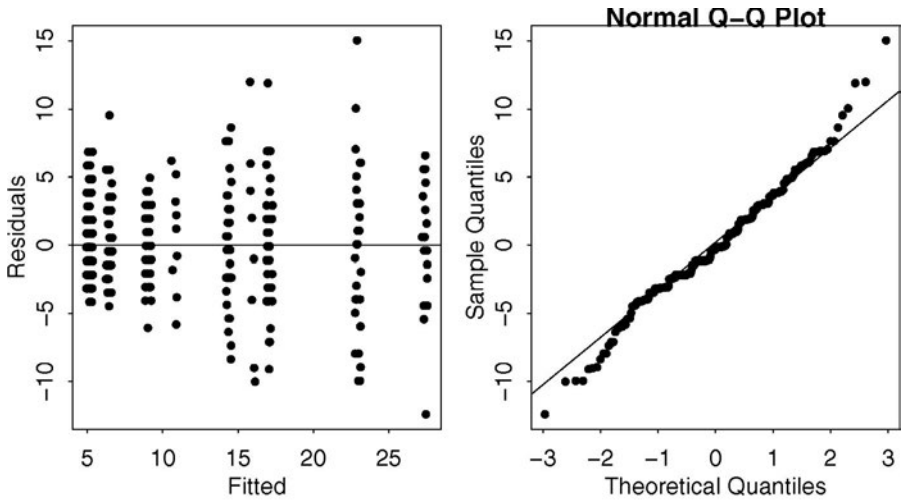


Figure 16.2 *Residuals and fitted values for the tree model of the Ozone data are shown in the left panel. A QQ plot of the residuals is shown in the right panel.*

x_2	1	2
	2	1
	x_1	

Table 16.1 *There are four data points arranged in a square. The number shows the value of y at that point.*

the vertical split will improve the fit at all. Both splits are required to get a better fit. We might need to look further ahead which will increase the computational cost. Furthermore, it’s not clear what the best choice of ϵ is for this strategy.

One general problem with model selection is that measures of fit such as the RSS (or deviance) usually improve as the complexity of the model increases. The measures tend to give a misleadingly optimistic impression of how well the model will predict future observations. A generic method of obtaining a better estimate of predictive ability is cross-validation (CV). For a given tree, leave out one observation, recalculate the tree and use that tree to predict the left-out observation. Repeat for all observations. For regression, this criterion would be:

$$\sum_{j=1}^n (y_j - \hat{f}_{(j)}(x_j))^2$$

where $\hat{f}_{(j)}(x_j)$ denotes the predicted value of the tree given the input x_j when case j is not used in the construction of the tree. For other types of trees, a different criterion would be used. For classification problems, it might be the deviance.

CV is a more realistic estimate of how the tree will perform in practice.

Leave-out-one cross-validation is computationally expensive so often k -fold cross-validation is used. The data is randomly divided into k roughly equal parts. We use $k - 1$ parts to predict the cases in the remaining part. We repeat k times, leaving out a different part each time. $k = 10$ is a typical choice. As well as being much less expensive computationally than the full leave-out-one method, it may even work better. One drawback is that the partition is random so that repeating the method will give different numerical results.

However, there may be very many possible trees if we consider all subsets of a large tree; cross-validation would just be too expensive. We need a method to reduce the set of trees to be considered to just those that are worth considering. This is where cost-complexity pruning is useful. We define a cost-complexity function for trees:

$$CC(Tree) = \sum_{\text{terminal nodes: } i} RSS_i + \lambda(\text{number of terminal nodes})$$

If λ is large, then the tree that minimizes this cost will be small and vice versa. Notice the similarity to AIC. We can determine the best tree of any given size by growing a large tree and then pruning it back. Given a tree of size n , we can determine the best tree of size $n - 1$ by considering all the possible ways of combining adjacent nodes. We pick the one that increases the fit criterion by the least amount. The strategy is akin to backward elimination in linear regression variable selection except that it can be shown that it generates the optimal sequence of trees of a given size.

We now use cross-validation to select from this sequence of trees. By default, `rpart` selects a tree size that may not be large enough to include all those trees we might want to consider. We force it to consider a larger tree and then examine the cross-validation criterion for all the subtrees. The parameter `cp` plays a similar role to the smoothing parameter in nonparametric regression and is defined as the ratio of λ to the RSS of the root tree (a tree with no branches). When we call `rpart` initially, it computes the whole sequence of trees and we merely need to use functions like `printcp` to examine the intermediate possibilities:

```
set.seed(123)
tmode <- rpart(O3 ~ ., ozone, cp=0.001)
printcp(tmode)
```

	CP	nsplit	rel	error	xerror	xstd
1	0.54570	0	1.000	1.005	0.0767	
2	0.07366	1	0.454	0.482	0.0427	
3	0.05354	2	0.381	0.427	0.0393	
4	0.02676	3	0.327	0.415	0.0392	
5	0.02328	4	0.300	0.403	0.0389	
6	0.02310	5	0.277	0.398	0.0387	
7	0.01532	6	0.254	0.396	0.0385	
8	0.01091	7	0.239	0.375	0.0343	
9	0.00707	8	0.228	0.353	0.0323	
10	0.00599	9	0.221	0.350	0.0346	
11	0.00593	10	0.215	0.354	0.0350	
12	0.00497	12	0.203	0.354	0.0350	
13	0.00480	15	0.188	0.353	0.0355	
14	0.00447	16	0.183	0.357	0.0359	
15	0.00319	17	0.179	0.358	0.0357	
16	0.00222	19	0.172	0.363	0.0365	
17	0.00207	20	0.170	0.362	0.0361	

18	0.00203	22	0.166	0.364	0.0361
19	0.00144	23	0.164	0.361	0.0361
20	0.00113	24	0.162	0.362	0.0361
21	0.00110	25	0.161	0.362	0.0361
22	0.00100	26	0.160	0.362	0.0361

In this table, we see the value of the `cp` parameter, the number of splits in the tree, the RSS of the tree divided by the RSS of the null tree. `xerror` denotes the cross-validated error which is also scaled by the RSS of the null tree. Since the partition of the data into 10 parts is random, this CV error is also random, which makes the given standard error useful. The random division also means that if you repeat this command, you will not get the same answer unless you use `set.seed`. We can select the size of the tree by minimizing the value of `xerror` and selecting the corresponding value of `CP`.

Because the selection method is quite dependent on the random division, another strategy for selecting the tree size can be considered. We can select the smallest tree with a CV error within one standard error of the minimum — in this case, $0.350 + 0.035 = 0.385$. So we would take the seven-split tree. We can illustrate this by plotting the CV error and a line showing one standard deviation above this value as shown in the first panel of Figure 16.3:

```
plotcp(tmod)
```

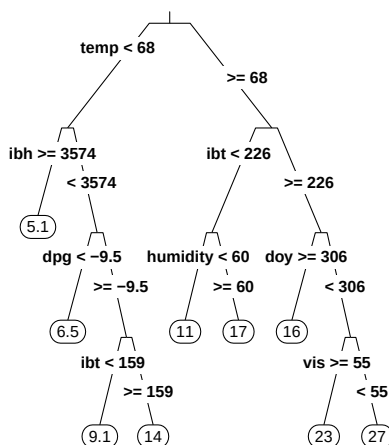
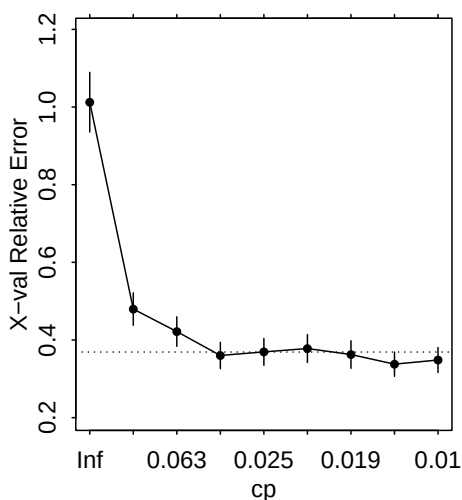


Figure 16.3 Cross-validation plot for ozone tree model shown in the left panel and chosen tree model shown in the right panel.

Notice that even the four-split tree comes close in terms of cross-validated error, so if we put a premium on simplicity, we might pick this tree. You can get some fancier output by using the `rpart.plot` package.

```
library(rpart.plot)
rpart.plot(tmod, type=3)
```

There are many options for the display—our chosen version is seen in the second panel of Figure 16.3. Let's compare the result to the earlier linear regression. We

achieved an R^2 of about 70% using only six parameters in the previous chapter. We can select a tree with five splits and hence effectively six parameters and compare them:

```
tmodr <- prune.rpart(tmod, 0.0154)
1-sum(residuals(tmodr)^2)/sum((ozone$O3-mean(ozone$O3))^2)
[1] 0.74603
```

We see that the tree model achieved a better fit than the equivalent linear model. Of course, it would be a mistake to generalize from this, but it is a good demonstration of the value of trees. A tree fit is piecewise constant over the regions defined by the partitions, so one might not expect a particularly good fit. However, we can see that it is not necessarily worse than linear regression.

Regression methods are used for two main purposes — prediction and explanation. Tree-based methods share the transparency of linear models in that it is relatively easy to see how they work and understand how they will predict for new inputs. However, they have two disadvantages relative to linear models. They lack the inferential apparatus of prediction intervals. They have discontinuities in the prediction at the partition boundaries but are constant elsewhere. For optimal prediction performance, more flexible methods tend to do better.

If the goal of the analysis is in explaining how the predictors are related to the response, the tree model has the apparent advantage of selecting which variables have a strong relationship to the response. However, these relationships are not always stable. For example, suppose we randomly divide our data in two and see what tree is formed. We choose the same size tree as our final choice above.

```
set.seed(123)
tmod <- rpart(O3 ~ ., ozone[sample(330, 165), ])
(tmodr <- prune.rpart(tmod, 0.0154))
n= 165
```

```
node), split, n, deviance, yval
  * denotes terminal node

1) root 165 9743.60 10.9520
 2) temp< 72.5 127 2836.00 7.9134
 4) ibt< 118 52 241.77 4.6538 *
 5) ibt>=118 75 1658.70 10.1730
 10) doy>=357 10 18.00 4.0000 *
 11) doy< 357 65 1201.00 11.1230
 22) ibh>=2327.5 24 170.50 8.7500 *
 23) ibh< 2327.5 41 816.24 12.5120
 46) temp< 65.5 29 317.86 11.0690 *
 47) temp>=65.5 12 292.00 16.0000 *
 3) temp>=72.5 38 1817.60 21.1050
 6) ibt< 226.5 15 227.60 15.6000 *
 7) ibt>=226.5 23 838.87 24.6960 *
```

Although the initial split is about the same, the variables chosen for subsequent splits are somewhat different. We have set the random seed so we will get the same results every time, but if you repeat this, you will get a different tree every time. This instability in variable selection shows that one should avoid overinterpreting the tree model on the full data. Of course, similar problems can be seen when using linear models but the results do tend to be more stable.

Tree models are not optimal for prediction or explanation purposes. Nevertheless, they do provide a contrasting approach to linear models that may provide additional insight into the structure of your data.

16.3 Random Forests

The experience with fitting tree models to random partitions of the data provides the inspiration for a method that builds on trees to form a forest. The *random forest* (RF) method, introduced by Breiman (2001a), uses bootstrap aggregating, known as *bagging*. For $b = 1, \dots, B$,

1. We draw a sample with replacement from (X, Y) to generate (X_b, Y_b) .
2. We fit a regression tree to (X_b, Y_b) .
3. For the set of cases not drawn in bootstrap sample (this will be about one third), we compute the mean squared error of prediction by inputting these predictor cases and comparing the predicted value to the observed value.

The latter step means that we have a measure of prediction performance that avoids the overfitting problem by not using data that was used in the construction of the given tree.

The B trees form the forest. Larger values of B are better although incremental improvement in performance levels off at some point. We will show later how we can be confident we have a sufficiently large B . We grow the trees as far as we can without going below a minimum of five cases per node. The trees in the forest will typically be larger than the one we would select as a single tree. New predictions can be made feeding the new predictor value into each of the trees in the forest and averaging the predictions made.

It has been observed that, for some datasets, certain predictors are chosen very frequently, meaning that there are strong correlations among the trees in the forest. To reduce this effect, at each node, a subsample of predictors is selected from which to choose a split. This ensures that every predictor has an opportunity to contribute to the prediction. The default choice of the subsample size is $\sqrt{p}$ where p is the number of predictors.

Let's fit and examine the default forest. We use the `randomForest` package of Liaw and Wiener (2002).

```
library(randomForest)
fmod <- randomForest(O3 ~ ., ozone)
plot(fmod, main="")
```

The plot of the returned model objects, as seen in the first panel of Figure 16.4, shows the mean squared error (MSE) of prediction as the number of trees B used increases. There is a rapid decrease in this MSE as we progress through smaller values of B but we can see that the improvement levels off after 100. We can see that the default choice of 500 trees is more than enough in this example. Since the forest was computed quite rapidly for this size of dataset, there is no incentive to economize. For much larger datasets, one might wish to reduce the number of trees to save time. This plot suggests how much economy may be reasonable.

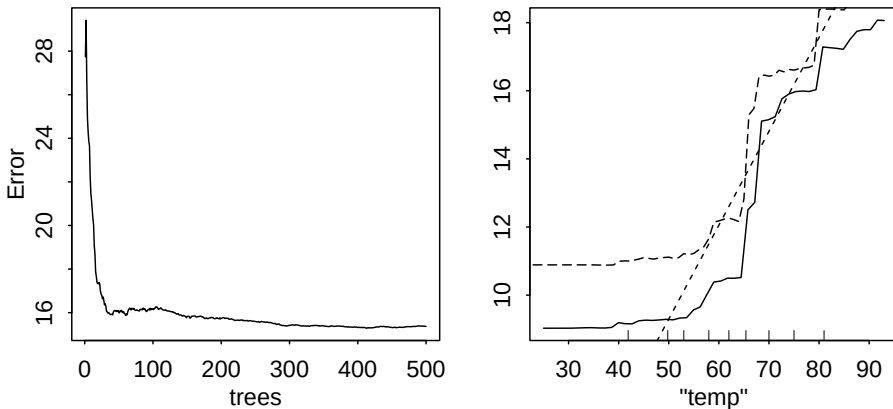


Figure 16.4 *MSE as a function of bootstrap sample size is shown on the left. Effect of temperature on ozone is shown on the right. The solid line is computed using the partial dependence method. The dashed line using the predicted RF response as temperature is varied and other predictors held at their means. The dotted line derives from a linear model.*

We must also choose the size of the subsample of predictors selected at each node (called `mtry` in the R command). This tuning parameter can have some impact on the performance of the forest. We can use cross-validation to choose from a range of values of `mtry`:

```
cvr <- rfcv(ozone[, -1], ozone[, 1], step=0.9)
cbind(nvars=cvr$n.var, MSE=cvr$error.cv)
```

	nvars	MSE
9	9	15.51
8	8	15.50
7	7	16.10
6	6	16.43
5	5	17.95
4	4	19.26
3	3	21.24
2	2	22.17
1	1	30.51

We have chosen the `step` fraction to be sufficiently large so that every subsample size is evaluated. We start with the full sample of $p = 9$ and then move on to $\text{step} \times p$ recursively, rounding at each stage. For larger datasets with more predictors, compute times may require us to be more economical by choosing a smaller `step` so that not too many subsample sizes are evaluated. We see that, in this example, a subsample of size 8 is suggested. However, since the performance of the full sample is hardly any different, we choose `mtry=9` for simplicity. Let's compute the R^2 for comparison to previous models:

```
fmod <- randomForest(O3 ~ ., ozone, mtry=9)
1-sum((fmod$predict-ozone$O3)^2)/sum((ozone$O3-mean(ozone$O3))^2)
[1] 0.7455
```

The resulting fit is similar to those seen previously.

Although we can readily make a prediction using the forest model, we lack the same interpretability provided by the tree. One idea is to fix the value of all but one

predictor at some typical value, say the median, and vary the chosen predictor over its range and observe how the predictor varies. Let's perform this calculation for the temperature predictor. We form a data frame where only the temperature predictor varies:

```
tgrid <- 20:100
meds <- apply(ozone, 2, median)[-match(c("O3", "temp"), names(ozone))]
tdf <- cbind(temp=tgrid, data.frame(t(meds)))
head(tdf)
```

	temp	vh	wind	humidity	ibh	dpg	ibt	vis	doy
1	20	5760	5	64	2112.5	24	167.5	120	205.5
2	21	5760	5	64	2112.5	24	167.5	120	205.5
3	22	5760	5	64	2112.5	24	167.5	120	205.5
4	23	5760	5	64	2112.5	24	167.5	120	205.5
5	24	5760	5	64	2112.5	24	167.5	120	205.5
6	25	5760	5	64	2112.5	24	167.5	120	205.5

Now we make predictions for all the cases in this data frame:

```
medfor <- predict(fmod, tdf)
```

We will plot these predictions but first we consider an alternative but similar approach called a *partial dependence plot* introduced by Friedman (1991). For each value of temperature on a grid, we compute the predicted value for *every* case in the dataset using the observed values of the predictors not including temperature. These predictions are then averaged. Thus this method requires n times as many calculations as our median-based method but forms an aggregate over the whole dataset. Such plots can be directly produced using:

```
partialPlot(fmod, ozone, "temp", main="")
```

We can add the median-based estimate of the effect:

```
lines(tgrid, medfor, lty=5)
```

Just for comparison purposes, we can make the same calculation using a linear model:

```
lmod <- lm(O3 ~ ., ozone)
lmpred <- predict(lmod, tdf)
lines(tgrid, lmpred, lty=2)
```

If you create all the partial dependence plots, you will find that temperature is the predictor exhibiting the strongest relationship with the response. Predictors *ibh* and *ibt* also demonstrate a lesser, but noticeable change in the predicted response over their range. In contrast, *wind* and *vh* show almost no relationship. This is an indication that, although random forests lack transparently interpretable regression coefficients, there is some indication of predictor importance. Such a measure of importance can be obtained as:

```
importance(fmod)
```

	IncNodePurity
vh	357.2
wind	177.8
humidity	891.3
temp	12820.6
ibh	1377.6
dpg	1034.4
ibt	2623.2
vis	714.0
doy	780.2

For regression-type forests, the importance is computed in the following way. For each bootstrap sample, the cases not chosen form the *out-of-bag* sample. We can use these to compute an MSE of prediction as before. Now for each predictor, we permute the values of that predictor in the out-of-bag sample while leaving the other predictors unchanged. We then recompute the MSE of prediction and compute the difference from unpermuted MSE. The permutation should destroy the relationship between that predictor and the response so if the predictor is important, the difference in the MSE will be large. We repeat this calculation over all the bootstrap samples and average the differences. This value is reported in the output of the `importance` above.

In this case, we see that temperature is by far the most important predictor. We also see which predictors we might safely discard, such as `vh` and `wind`. This method has no test for significance or criterion-based variable selection, but it does indicate the order in which we might consider discarding variables.

Random forests have demonstrated strong prediction performance in practice across a wide range of datasets. If the goal of your analysis is prediction, then the random forest is a good choice. In contrast, if your goal is to explain the relationship between the predictors and the response, random forests may provide some insight but they lack the full power of linear or additive models.

16.4 Classification Trees

Trees can be used for several different types of response data. For the regression tree, we computed the mean within each partition. This is just the null model for a regression. We can extend the tree method to other types of response by fitting an appropriate null model on each partition. For example, we can extend the idea to binomial, multinomial, Poisson and survival data by using a deviance, instead of the RSS, as a criterion.

Classification trees work similarly to regression trees except the residual sum of squares is no longer a suitable criterion for splitting the nodes. The splits should divide the observations within a node so that the class types within a split are mostly of one kind (or failing that, just a few kinds). We can measure the *purity* of the node with several possible measures. Let n_{ik} be the number of observations of type k within terminal node i and p_{ik} be the observed proportion of type k within node i . Let D_i be the measure for node i so that the total measure is $\sum D_i$. There are several choices for D_i :

1. Deviance:

$$D_i = -2 \sum_k n_{ik} \log p_{ik}$$

2. Entropy:

$$D_i = - \sum_k p_{ik} \log p_{ik}$$

3. Gini index:

$$D_i = 1 - \sum_k p_{ik}^2$$

All these measures share the characteristic that they are minimized when all members of the node are of the same type. The `rpart` function uses the Gini index by default.

We illustrate the classification tree method in a problem involving the identification of the sex and species of an historical specimen of kangaroo. We have some training data consisting of 148 cases with the following variables: there are three possible species, *Giganteus*, *Melanops* and *Fuliginosus*, the sex of the animal and 18 skull measurements. The data were published in Andrews and Herzberg (1985). The historical specimen is from the Rijksmuseum van Natuurlijke Historie in Leiden which had the following skull measurements in the same order as in the data:

```
1115 NA 748 182 NA NA 178 311 756 226 NA NA NA 48 1009 NA 204 593
```

We have a choice in how we model the response. One possibility is to form a six-level response representing all possible combinations of sex and species. Another approach is to form separate trees for identifying the sex and the species. We take the latter approach below, focusing on the species. This choice is motivated by the belief that different features are likely to discriminate the sex and the species so that attempting to model them both in the same tree might result in a larger, more complex tree that might be less powerful than two smaller trees. Even so, it would be worth trying the first approach although we shall not do so here. We start by reading in and specifying the museum case:

```
data(kanga, package="faraway")
x0 <- c(1115, NA, 748, 182, NA, NA, 178, 311, 756, 226, NA, NA, NA, 48, 1009, NA
      ↪ , 204, 593)
```

We have missing values for the case to be classified. We have two options. We can build a tree model that will classify if there are missing values in the input or we can build a tree model that uses only variables that are observed. If we believe that the missing values were in some way informative, the first choice would be sensible. In this particular case, that does not seem plausible, so the latter approach is preferred. However, if we want to build a model that could be used for future unspecified cases, then we would have to deal directly with the missing values. For this special purpose situation, where we want to classify one particular kangaroo, this is not a concern.

We exclude all variables that are missing in the test case. We drop sex since we will not be modeling it yet. We form a convenient data frame:

```
kanga <- kanga[, c(T, F, !is.na(x0))]
kanga[1:2, ]
```

	species	basilar.length	palate.length	palate.width	squamosal.depth
1	giganteus	1312	882	NA	180
2	giganteus	1439	985	230	150
	lacrymal.width	zygomatic.width	orbital.width	foramina.length	
1	394	782	249	88	
2	416	824	233	100	
	mandible.length	mandible.depth	ramus.height		
	10861	179	591		
	11582	181	643		

We still have missing values in the training set. We have a number of options:

1. Build a tree model that discretizes the predictors into factors and then treats missing values as another level of the factors. This might be appropriate if we think missing values are informative in some way. Information would be lost in the discretization. For this data, we have no reason to believe that the data is not missing

at random and furthermore we have already decided to ignore the missing values in the test case.

2. Fill in or estimate the missing values and then build a tree. We could use missing data fill-in methods as used in other regression problems. This is not easy to implement and there are concerns about the bias caused by such methods.
3. The tree-fitting algorithm can handle missing values naturally. If a value for some case is not available, then it is simply excluded from the criterion. When we want to classify a new case with missing values, we follow the tree down until we reach a split that involves a missing value in our new case and take the majority verdict in that node. A more complicated approach is to allow a second-choice variable for splitting at a node called a *surrogate split*. Information on the surrogate splits may be obtained by using the `summary` command on the tree object.
4. Leave out the missing cases entirely.

We first check where the missing values occur:

```
apply(kanga, 2, function(x) sum(is.na(x)))
```

species	basilar.length	palate.length	palate.width
0	1	1	24
squamosal.depth	lacrymal.width	zygomatic.width	orbital.width
1	0	1	0
foramina.length	mandible.length	mandible.depth	ramus.height
0	12	0	0

We observe that the majority of missing values occur in just two variables: mandible length and palate width. Suppose we throw out those variables and then remove the remaining missing cases. We compute the pairwise correlation of these variables with the other variables.

```
round(cor(kanga[, -1], use="pairwise.complete.obs")[, c(3, 9)], 2)
```

	palate.width	mandible.length
basilar.length	0.77	0.98
palate.length	0.81	0.98
palate.width	1.00	0.81
squamosal.depth	0.69	0.80
lacrymal.width	0.77	0.92
zygomatic.width	0.78	0.92
orbital.width	0.12	0.25
foramina.length	0.19	0.23
mandible.length	0.81	1.00
mandible.depth	0.62	0.85
ramus.height	0.73	0.94

We see that these two variables are highly correlated with other variables in the data. We claim that there is not much additional information in these two variables and we can reasonably discard them. We do this and then remove the remaining missing cases:

```
newko <- na.omit(kanga[, -c(4, 10)])
```

```
dim(newko)
```

```
[1] 144 10
```

After excluding these two variables, we only lose four cases more by throwing out all the missing value cases. Alternatively, suppose we just throw out the missing value cases on the original data:

```
dim(na.omit(kanga))
```

```
[1] 112 12
```

We would lose 36 cases by simply throwing out all the missing values. Removing a combination of variables and cases seems a better choice for this data.

We should also plot the data to see how the classes separate. An example of such a plot is:

```
ggplot(newko, aes(x=zygomatic.width, y=foramina.length, shape=species)) +
  geom_point() + theme(legend.position = "top", legend.direction =
    "horizontal", legend.title=element_blank())
```

We see in the left panel of Figure 16.5 that the classes do not separate well, at least for these two variables. We now fit a classification tree as follows: because the response

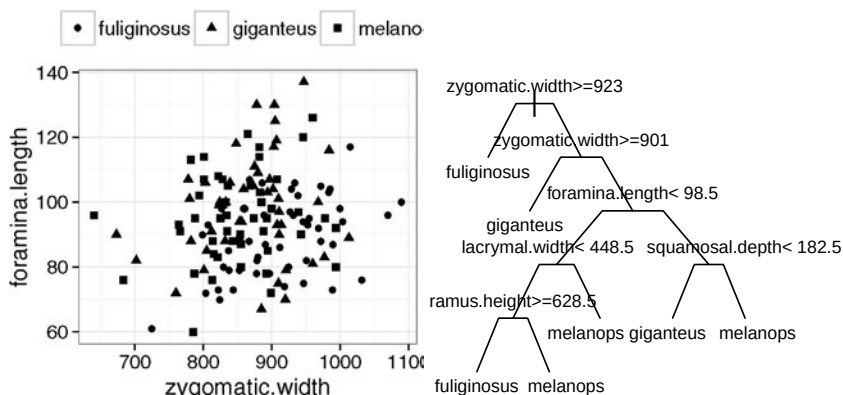


Figure 16.5 *Historical kangaroo tree model. The left panel shows the three species as they vary with two of the measurements. The right panel shows the chosen tree.*

is a factor, classification rather than regression is automatically used. Gini's index is the default choice of criterion. Here we specify a smaller value of the complexity parameter `cp` than the default, so that larger trees are also considered:

```
set.seed(123)
kt <- rpart(species ~ ., data=newko, cp=0.001)
printcp(kt)
```

```
Root node error: 95/144 = 0.66
```

```
n= 144
```

	CP	nsplit	rel error	xerror	xstd
1	0.179	0	1.00	1.20	0.051
2	0.105	1	0.82	0.97	0.061
3	0.050	2	0.72	0.86	0.063
4	0.021	6	0.52	0.82	0.063
5	0.011	7	0.49	0.84	0.063
6	0.001	8	0.48	0.87	0.062

The cross-validated error (expressed in relative terms in the `rel error` column) reaches a minimum for the six-split tree. We select this tree:

```
(ktp <- prune(kt, cp=0.0211))
```

```
n= 144
```

```
node), split, n, loss, yval, (yprob)
  * denotes terminal node
```

```
1) root 144 95 fuliginosus (0.34028 0.33333 0.32639)
  2) zygomatic.width>=923 37 13 fuliginosus (0.64865 0.16216 0.18919) *
  3) zygomatic.width< 923 107 65 giganteus (0.23364 0.39252 0.37383)
    6) zygomatic.width>=901 16 3 giganteus (0.12500 0.81250 0.06250) *
    7) zygomatic.width< 901 91 52 melanops (0.25275 0.31868 0.42857)
      14) foramina.length< 98.5 58 33 melanops (0.36207 0.20690 0.43103)
        28) lacrymal.width< 448.5 50 29 fuliginosus (0.42000 0.24000 0.34000)
          56) ramus.height>=628.5 33 14 fuliginosus (0.57576 0.18182 0.24242) *
          57) ramus.height< 628.5 17 8 melanops (0.11765 0.35294 0.52941) *
        29) lacrymal.width>=448.5 8 0 melanops (0.00000 0.00000 1.00000) *
      15) foramina.length>=98.5 33 16 giganteus (0.06061 0.51515 0.42424)
        30) squamosal.depth< 182.5 26 10 giganteus (0.07692 0.61538 0.30769) *
        31) squamosal.depth>=182.5 7 1 melanops (0.00000 0.14286 0.85714) *
```

```
plot(ktp,compress=T,uniform=T,branch=0.4)
text(ktp)
```

This tree is not particularly successful as the relative error is estimated as 80% of just guessing the species. Some of the terminal nodes are quite pure, for example, #29 and #31, while others retain much uncertainty, for example, #56 and #57. We now compute the misclassification error:

```
(tt <- table(actual=newko$species, predicted=predict(ktp, type="class"
  ↪ )))
```

	predicted		
actual	fuliginosus	giganteus	melanops
fuliginosus	43	4	2
giganteus	12	29	7
melanops	15	9	23

```
1-sum(diag(tt))/sum(tt)
[1] 0.34028
```

We see that the error rate is 34%. We might hope to do better. We see that we can generally correctly identify *fuliginosus*, but we are more likely to be in error in distinguishing *melanops* and *giganteus*.

A look at the left panel of Figure 16.5 explains why we may have difficulty in classification. Any single measure will reflect mostly the overall size of the skull. For example, suppose we wanted to distinguish male human skulls from female human skulls. Most interesting measures will correlate strongly with size. If we just use one measure, then the rule will likely be: if the measure is small, then pick female; if it is large, pick male. This cannot be expected to work particularly well. There is something about the *relative* dimensions of the skulls that ought to be more informative.

One possibility is to allow splits on linear combinations of variables. This is allowed in some classification tree software implementations. An alternative idea is to apply the method to the principal component scores rather than the raw data. Principal components (PC) seek out the main directions of variation in the data and might generate more effective predictors for classification in this example:

```
pck <- princomp(newko[, -1])
pcdf <- data.frame(species=newko$species, pck$scores)
kt <- rpart(species ~ ., pcdf, cp=0.001)
printcp(kt)
```

Root node error: 95/144 = 0.66

n= 144

	CP	nsplit	rel error	xerror	xstd
1	0.400	0	1.00	1.18	0.053
2	0.179	1	0.60	0.76	0.063
3	0.042	2	0.42	0.52	0.060
4	0.011	3	0.38	0.54	0.060
5	0.001	5	0.36	0.61	0.062

We find a significantly smaller relative CV error (0.52). Our chosen tree is:

```
(ktp <- prune.rpart(kt,0.0421))
```

n= 144

```
node), split, n, loss, yval, (yprob)
* denotes terminal node
```

```
1) root 144 95 fuliginosus (0.34028 0.33333 0.32639)
  2) Comp.2< -15.13 49 8 fuliginosus (0.83673 0.04082 0.12245) *
    3) Comp.2>=-15.13 95 49 giganteus (0.08421 0.48421 0.43158)
      6) Comp.4>=-9.513 63 24 giganteus (0.11111 0.61905 0.26984)
        12) Comp.3>=-19 55 17 giganteus (0.09091 0.69091 0.21818) *
          13) Comp.3< -19 8 3 melanops (0.25000 0.12500 0.62500) *
            7) Comp.4< -9.513 32 8 melanops (0.03125 0.21875 0.75000) *
```

It is interesting that the first PC is not used. Typically, the first PC represents an overall average or total size. Other PCs represent contrasts between variables which would describe shape features in this case. We can also compute the error rate as before:

```
(tt <- table(newko$species,predict(ktp,type="class")))
```

	fuliginosus	giganteus	melanops
fuliginosus	41	5	3
giganteus	2	38	8
melanops	6	12	29

```
1-sum(diag(tt))/sum(tt)
```

```
[1] 0.25
```

We see that the error rate has been reduced to 25%. It would be worth considering other combinations of predictors in an attempt to reduce the error rate further.

Before we can predict the test case, we need to do some work to remove the missing values and unused variables and apply the principal component transformation:

```
nx0 <- x0[!is.na(x0)]
nx0 <- nx0[-c(3,9)]
nx0 <- (nx0-pck$center)/pck$scale
ndf <- data.frame(nx0 %%% pck$loadings)
predict(ktp, ndf)
```

	fuliginosus	giganteus	melanops
1	0.83673	0.040816	0.12245

We see that the test case is classified as *fuliginosus*, which agrees with the experts.

Given the structure of this dataset, with strongly correlated predictors, we might expect linear discriminant analysis, as explained in Section 7.2, to perform well. It is simple to implement:

```
library(MASS)
ldamod <- lda(species ~ ., newko)
(tt <- table(newko$species,predict(ldamod)$class))
```

	fuliginosus	giganteus	melanops
fuliginosus	42	6	1
giganteus	5	32	11
melanops	3	14	30

```
1-sum(diag(tt))/sum(tt)
```

```
[1] 0.27778
```

We see that the default classification performance is reasonably good compared to trees.

It is a mistake to generalize from a single dataset. Different classification methods are likely to do well for certain kinds of data but perform poorly in others. It's always a good idea to try more than one.

16.5 Classification Using Forests

Random forests are also a natural extension to classification trees. These can be applied in much the same way as in the regression case but now we use a measure of node purity such as the Gini index in place of MSE as a measure of success. We demonstrate the random forest method on the kangaroo skull data. We start by selecting the subsample size:

```
cvr <- rfcv(newko[,-1],newko[,1],step=0.9)
cbind(nvars=cvr$n.var,error.rate=cvr$error.cv)
```

	nvars	error.rate
9	9	0.45139
8	8	0.43750
7	7	0.43750
6	6	0.43750
5	5	0.47222
4	4	0.55556
3	3	0.55556
2	2	0.59028
1	1	0.59028

We see that we can go as low as six for the sub-sample size. All things be equal, a small subsample size will be faster. Now given this choice:

```
fmod <- randomForest(species ~ ., newko, mtry=6)
(tt <- table(actual=newko$species,predicted=predict(fmod)))
```

	predicted		
actual	fuliginosus	giganteus	melanops
fuliginosus	38	5	6
giganteus	6	21	21
melanops	10	21	16

```
1-sum(diag(tt))/sum(tt)
```

```
[1] 0.47917
```

The performance is quite poor. As before, we might attempt to improve matters using principal components on the predictors:

```
pck <- princomp(newko[,-1])
pcdf <- data.frame(species=newko$species,pck$scores)
fmod <- randomForest(species ~ ., pcdf, mtry=6)
tt <- table(actual=newko$species,predicted=predict(fmod))
1-sum(diag(tt))/sum(tt)
```

```
[1] 0.33333
```

The result is not impressive but random forests have a strong track record in classification across a wide range of problems so do not let this single poor outcome discourage you from using this method.

Exercises

- Four hundred three African Americans were interviewed in a study to understand the prevalence of obesity, diabetes and other cardiovascular risk factors in central Virginia. Data is presented in `diabetes`. In this question we build a regression tree-based model for predicting glycosolated hemoglobin (`glyhb`) in terms of the other relevant variables.
 - Plot the response against each of the predictors and comment on the apparent strength of the relationships observed.
 - Investigate the pattern of missing values in the data. By eliminating a combination of rows and columns, produce a reduced dataset that contains no missing values. Try to make the reduction as small as possible.
 - Fit a linear model with main effects for all the predictors. Which predictors appear to be most important? Make a plot of the residuals and fitted values from this model and comment.
 - Fit the default tree. From the output, answer the following questions: How many observations had `stab.glu < 158`? What is the mean response for these observations? What characterizes the largest terminal node in the tree? What is the mean response in this node?
 - Make a plot of the tree. What feature of the plot reveals the most important predictor?
 - Plot the residuals against the fitted values for this tree. Comment.
 - Select the optimal tree using cross-validation. What would be the smallest tree that could reasonably be used?
 - Predict the response for an individual with these predictor values:

id	chol	stab.glu	hdl	ratio	location	age	gender	height
1004	213		72	58	3.3	Buckingham	56	female
								64
weight	frame	bp.1s	bp.1d	bp.2s	bp.2d	waist	hip	time.ppn
131	medium	108	55	NA	NA	30	40	720
 - Glycosolated hemoglobin greater than 7.0 is usually taken as a positive diagnosis of diabetes. Build the default classification tree for the diagnosis of diabetes and compare this model to the corresponding regression tree. Which predictors are most useful?
- Refer to the `pima` dataset described in Question 3 of Chapter 15. If you have not done so already, answer parts (a) and (b) of that question before proceeding with the rest of this question.
 - Fit the default tree model with the result of the diabetes test as the response and all the other variables as predictors using the training set. Make a nice plot of the tree and comment on the shape of the tree. Is it broad or deep?

- (b) What fraction of cases in the test set are correctly classified according to this model?
 - (c) Use cross-validation to select the optimal tree size. For the selected model, check the performance on the test set.
 - (d) Fit the default random forest model to the training set. Use this to classify the cases in the test set. How many are correctly classified?
 - (e) Use the subsample size selection method for the random forest. For the selected model, evaluate the performance on the test set.
3. The dataset `wbca` is described in Question 2 of Chapter 2. You may find it helpful to repeat the plotting part of that question first.
- (a) Without modifying the data, fit a tree model with `Class` as the response and the other nine variables as predictors. Now change `Class` to a factor and refit the tree. What is the difference between these two trees and which one should we prefer?
 - (b) Use the default (10-way) cross-validation to select a tree size. Now repeat the cross-validation. Explain why the outcome is different. Now use leave-out-one cross-validation (set the argument `xval` equal to the sample size). Does this calculation change if it is repeated? What model is chosen by this latter method?
 - (c) Using the last model chosen in the previous question, determine how many cases in the data are correctly classified using the model. (Suppose a cancer is classified as benign if $p > 0.5$ and malignant if $p < 0.5$.)
 - (d) Compute the receiver operating characteristic (ROC) plot by varying the 0.5 threshold used in the previous question.
 - (e) Pick out every third observation as a test set. Use the remainder as a training set with which to estimate the model. Evaluate the performance on the test set. Compare the results to (c). Which classification rate is a more reliable estimate of future performance?
4. The dataset `uswages` is drawn as a sample from the Current Population Survey in 1988.
- (a) Plot the `wage` response with each of the remaining variables as potential predictors. Try a log-scale for the response. Does this reveal more about the relationships in the data?
 - (b) Fit the default tree regression model to predict `wage` using untransformed versions of all the predictors. Now transform years of experience using the function $\log(x + 3)$. (The $+3$ is to cancel out some negative values.) Does this change the fit of the model? Explain.
 - (c) Now fit a model with a logged response. Compare this to the previous model and explain why it is different. Compare the RSS from both models — you will need to back-transform the fitted values from the logged model for comparability.

- (d) Predict the response from the untransformed tree model for each year of education ranging from 0 up to 18 with the other predictors held at their median values. Plot the observed data on wages and years of education and display your predictions on top of this plot. Now fit a standard linear model to the same variables, compute the predictions in the same manner and also display on the same plot. Compare the two sets of predictions.
5. The `dvisits` data comes from the Australian Health Survey of 1977–1978 and consist of 5190 single adults where young and old have been oversampled.
- (a) Build a Poisson tree model with `doctorco` as the response and `sex`, `age`, `agesq`, `income`, `levyplus`, `freepoor`, `freerepa`, `illness`, `actdays`, `hscore`, `chcond1` and `chcond2` as possible predictor variables. Consult the `rpart` documentation for how to specify a Poisson response. What size tree might be the minimum acceptable?
 - (b) Make a plot of the selected tree and interpret. How do the predictors explain the response.
 - (c) For the last person in the dataset, compute the predicted probability distribution for their visits to the doctor, i.e., give the probability they visit 0, 1, 2 etc. times.
 - (d) Fit a random forest model to the same variables. You will need to use the regression method. Construct the same predicted probabilities as in the previous question.
 - (e) Assess the importance of each variable in the forest model. Make partial dependence plots of the most and the least important predictors. Compare the two.

This page intentionally left blank